수치해석: 최적화

(에너지 및 동력전달 해석, 산업공학, 딥러닝)

청주대학교 항공기계공학과 조 성 욱

청주대학교 부설
항공모빌리티 연구소
Institute for Air Mobility

충북 드론·UAM 연구센터
Chungbuk UAM·Drone Research Center

◎ 본 자료는 “공학도를 위한 수치해석(numerical methods for engineers), 한빛아카데미, 2022”, “파이썬을 이용한 수치해석, 씨아이알, 2020”, “수치해석바이블, 한빛미디어, 2018”을 참고하여 작성되었습니다. 이에 감사드립니다.

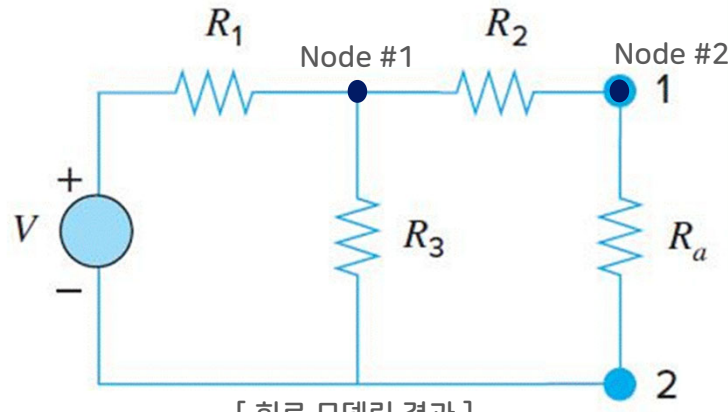
전기전자: 회로 내 최대 전력 전달

- 가변저항 운용을 통한 회로 내 전력변화 모델링
  - ➢ 주어진 회로를 직류 저항회로로 단순화 → 가변저항 위치 결정
  - ➢ 옴의 법칙과 키르히호프의 법칙으로 회로 모델링

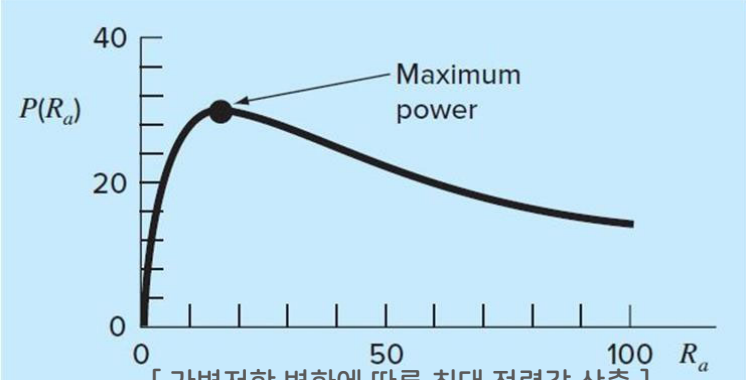
$$\frac{0+V-v_1}{R_1}=\frac{v_1-0}{R_3}+\frac{v_1-v_2}{R_2} \qquad \frac{v_1-v_2}{R_2}=\frac{v_2-0}{R_a}$$

  - ➢ 전력 공식 적용 및 최적 가변저항값 계산

$$P(R_a)=\frac{1}{R_a}\left[\frac{VR_3R_a}{R_1(R_a+R_2+R_3)+R_3R_a+R_3R_2}\right]^2$$



[회로 모델링 결과]

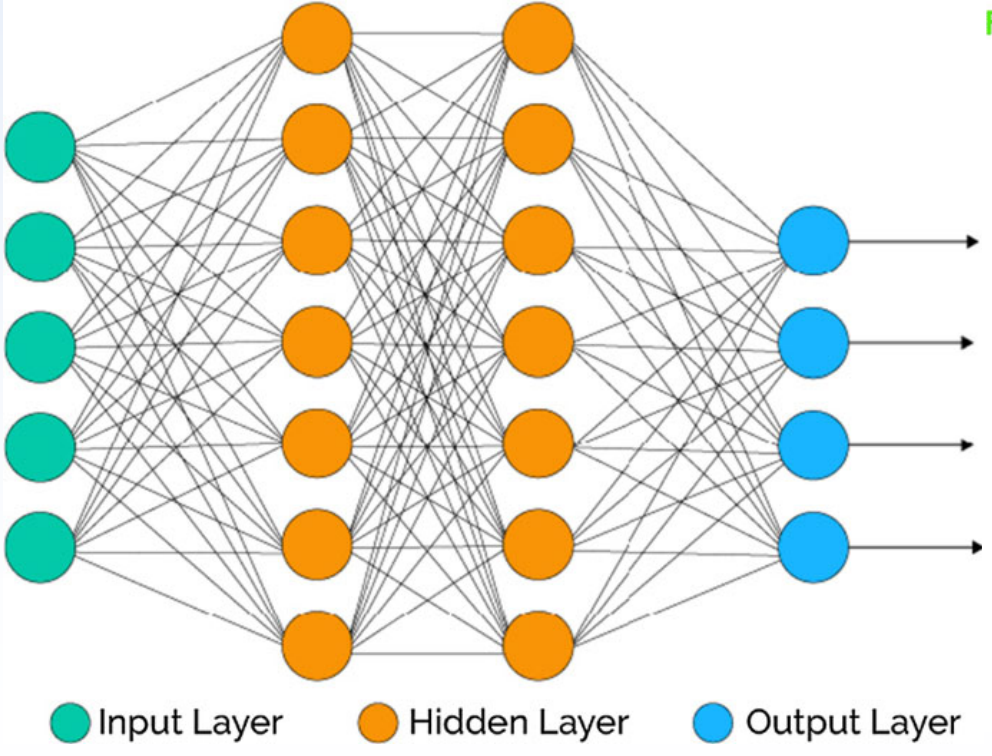


[가변저항 변화에 따른 최대 전력값 산출]

$$\frac{dP}{dR_a}=\frac{-2R_3^2R_aV^2(R_1+R_3)}{\{R_1(R_2+R_3+R_a)+R_2R_3+R_3R_a\}^3}+\frac{R_3^2V^2}{\{R_1(R_2+R_3+R_a)+R_2R_3+R_3R_a\}^2}=0$$

영상처리: 딥러닝 기반 객체 인식

- 적층된 신경망 구조의 가중치값을 결정할 때, 최적화 알고리즘 적용



[딥러닝 모델 구조 예시]

Forward Pass

- **Hidden Layer**

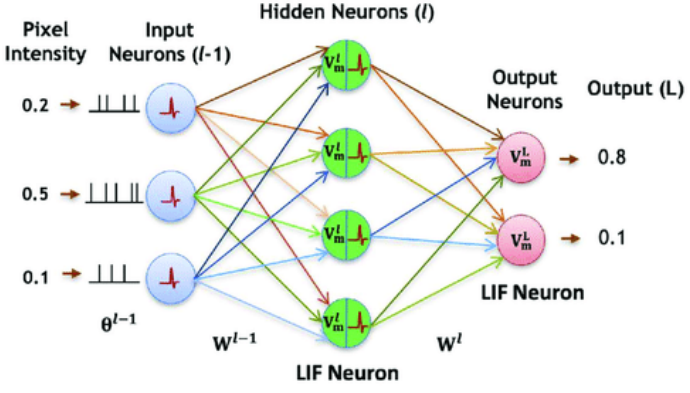
Total input current : $net^l(t) = \sum_{i=1}^{i=n^{l-1}} W_i^{l-1} X_i^{l-1},\ X_i^{l-1} = \sum_t \sum_k \theta_i^{l-1}(t-t_k)$

Activation of Neurons : $a_{LIF}^l(t) = \sum_t \sum_k \theta^l(t-t_k)$

- **Final Layer**

$net^L(t) = \sum_{i=1}^{i=n^l} W_i^l X_i^l,\ X_i^l = \sum_t \sum_k \theta_i^l(t-t_k)$

$a_{LIF}^L(net^L) = output = \frac{1}{T} V_{mem}^L(t)$



Backward Pass

- **Hidden Layer**

Error Gradient : $\delta^l = ((W^l)^{Tr} * \delta^L).a_{LIF}^l{}'(net^l)$

$a_{LIF}^l{}'(net^l) = \frac{1}{V_{th}}\left(1 + \frac{1}{\gamma} f'(t)\right)$

- **Final Layer**

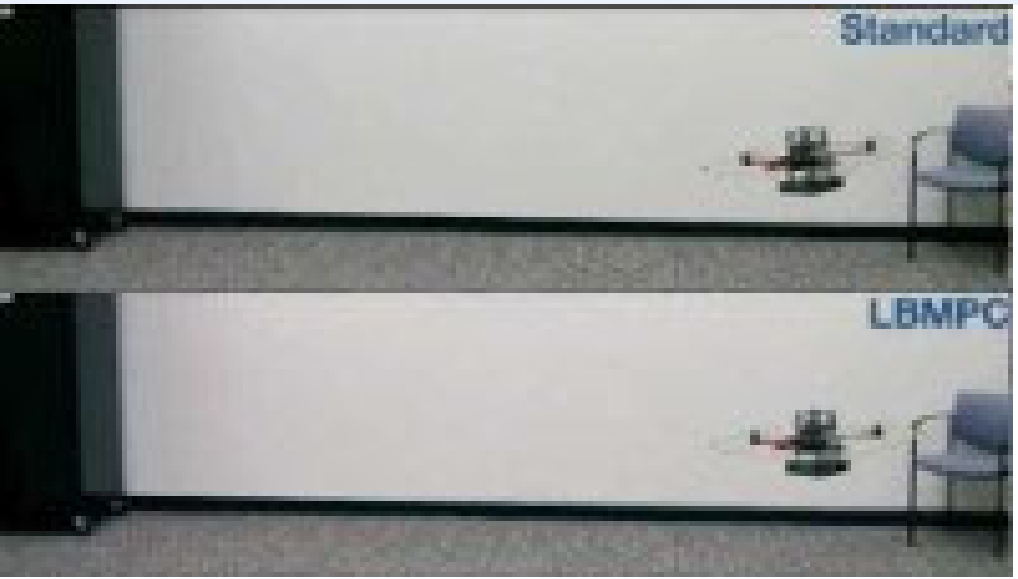
$\delta^L = (output - label).a_{LIF}^L{}'(net^L) = e.\frac{1}{T}$

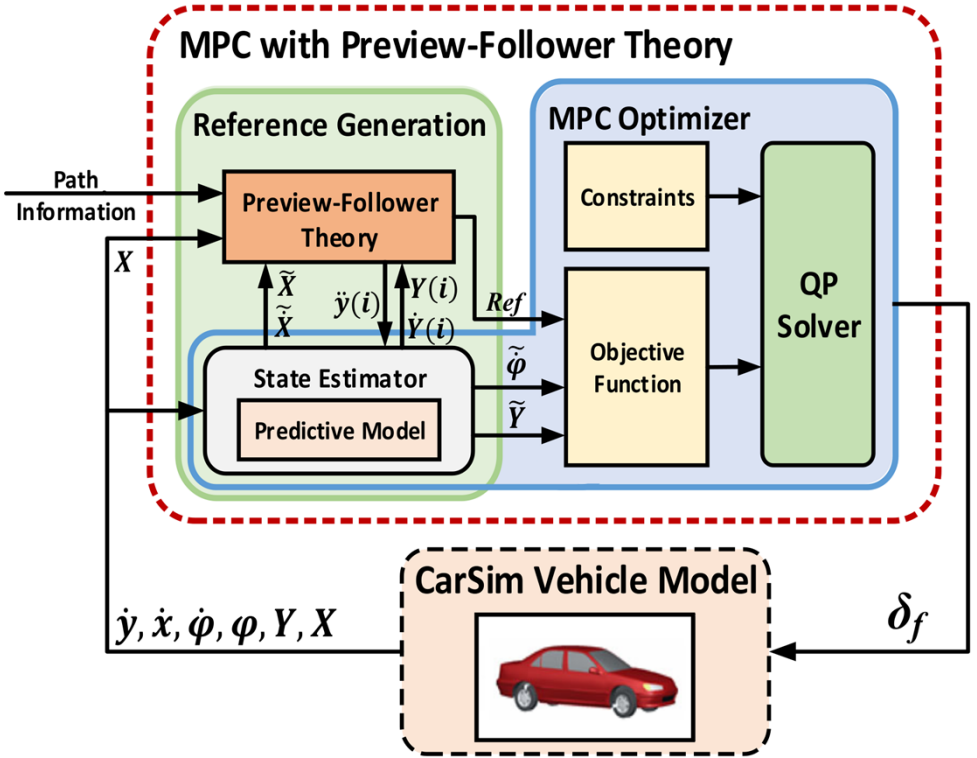
Output Error (e) = output − label

[딥러닝 모델의 가중치를 결정하기 위한 전방/후방 계산 예시]

무인이동체 제어: 최적 제어명령 또는 유도제어궤적 산출

- 내부 자세제어, 외부 유도제어, 임무수행에 필요한 최적의 결과값을 산출하는데 최적화 알고리즘이 활용됨





$$
\begin{aligned}
\underset{u(\cdot)}{\text{minimize}} \quad & \int_t^{t+N} l(x(\tau), u(\tau))\, d\tau \\
\text{subject to} \quad & x(t) = x_0 \\
& \left.\begin{aligned} \dot{x}(\tau) &= f(x(\tau), u(\tau)) \\ x(\tau) &\in \mathcal{X} \\ u(\tau) &\in \mathcal{U} \end{aligned}\right\} \forall \tau \in [t, t+N]
\end{aligned}
$$

[자율 비행/주행을 달성하기 위한 최적의 경로 또는 자세제어 명령 산출 관련 예시]

허용된 자원의 한계 내에서 주어진 요구사항을 만족시키면서 최선의 결과를 얻는 과정

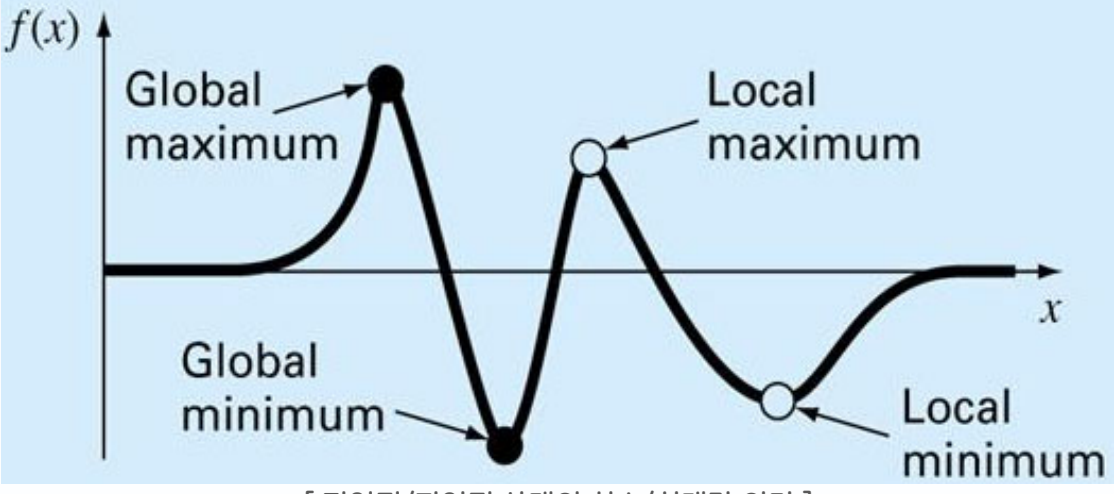
- 수학/물리적 측면: 주어진 또는 모델링한 수학/물리적 함수의 최대/최소값을 찾는 방법
  - ➢ 특정한 집합 내에서 정의된 실수값, 함수, 정수 등에 대해 그 값이 최대나 최소가 되는 상태를 해석하는 문제
    - ▪ 제약조건에 따라, 비구속(unconstraint)/구속(constraint) 상태로 구분됨
    - ▪ 공학적/경제적/사회적 등 다양한 측면에서 이미 최적화 문제를 설정 및 해결하고 있음
      - 최소 중량, 최대 강도/강성을 동시에 고려
      - 양력 최대, 항력 최소를 동시에 고려
      - 최소 비용, 최대 효과 동시에 고려
    - ▪ 항상 주어진 상태의 전역적 최대/최소를 구할 수 없음
      - 이 점을 고려하여 후처리 알고리즘 구성해야 함
    - ▪ 이를 극복하기 위한 연구개발이 지속되고 있음
- 비선형 방정식 해 구하기 vs. 최적화

$$f(x)=0 \quad vs. \quad f'(x)=0$$

(위 조건을 만족하는 x값 찾기)

$$\min.: f''(x)>0$$
$$\max.: f''(x)<0$$

(f(x)의 최대값/최소값을 나타내는 극값을 가리치는 x값 찾기)



[전역적/지역적 상태의 최소/최대값 의미]

비구속/구속조건 하의 대표적인 최적화 기법 이론 학습 및 실습을 통한 활용

- 비용함수(cost function) 또는 목적함수(object function)를 설정 또는 설계
  - ➢ 등식 또는 부등식으로 설정된 제약조건 만족 필요
  - ➢ 제약조건이 없는 경우, $F(\boldsymbol{x})$가 최소인 점의 좌표는 비선형 연립방정식 $\nabla F(\boldsymbol{x})$의 해로 계산됨
- **초기값 지정 이후 반복계산을 통해 주어진 함수의 최소값 결정 → 공학적 설계 시 가장 중요한 행위**
  - ➢ 일반적인 목적함수 형태

$$F^*(\boldsymbol{x},\lambda) = F(\boldsymbol{x}) + \sum_{i=1}^{m} \lambda_i g_i(\boldsymbol{x})$$

$$P(\boldsymbol{x}) = \sum_{i=1}^{m} [g_i(\boldsymbol{x})]^2 + \sum_{j=1}^{n} \{\max[0, h_j(\boldsymbol{x})]\}^2$$

$$\nabla F^*(\boldsymbol{x}) = \boldsymbol{0}, \quad g_i(\boldsymbol{x}) = \boldsymbol{0} \quad (i = 1, 2, \cdots, m)$$

- λ_i: Lagrangian multipliers
  - λ가 작으면, 더 큰 공간에서 처리하므로, 빠른 최적화 가능 (제약조건 위배 가능)
  - λ가 크면, 나쁜 조건에서 처리되는 결과가 나올 수 있지만, 제약조건은 엄격하게 적용
  - 작은 λ로 최적화 프로그램을 실행하는 것을 권장
  - 제약조건이 크게 다른 경우 최적화 과정이 어려울 수 있음
  - 위배한 제약조건을 조정하여 경감할 수 있음(제약조건식에 적절한 상수를 곱함)

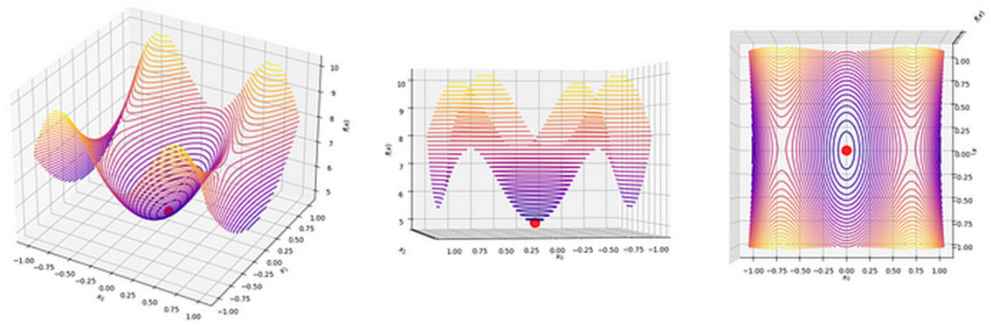
Table 3. Optimal Design of Mission Performances

임무	임무조건	설계변수	구속조건	목적함수
이륙 (take-off)	이륙속도 = $1.1\,V_{stall}$ 고도 = $0\,\mathrm{ft}$ 받음각	조종면의 변위		양력계수 최대화
상승/하강 (climb/descent)	시작속도 종료속도 시작고도 종료고도 엔진출력	받음각 조종면의 변위		양항비 최대화
순항 1 (cruise 1)	순항거리	순항속도 순항고도 받음각 조종면의 변위 엔진출력	모멘트 계수 = 0	연료소비량 최소화
순항 2 (cruise 2)	순항거리 순항고도	순항속도 받음각 조종면의 변위 엔진출력	모멘트 계수 = 0	연료소비량 최소화
순항 3 (cruise 3)	순항거리 순항속도	순항고도 받음각 조종면의 변위 엔진출력	모멘트 계수 = 0	연료소비량 최소화
순항 4 (cruise 4)	순항거리 순항속도 순항고도	받음각 조종면의 변위 엔진출력	모멘트 계수 = 0	연료소비량 최소화
가속 (acceleration)	시작속도 종료속도 임무고도	받음각 조종면의 변위 엔진출력	모멘트 계수 = 0	가속시간 최소화
전투 (combat)	임무속도 임무고도 엔진출력 = 2.0	받음각 조종면의 변위		양항비 최대화
선회 (loiter)	임무시간	임무속도 임무고도 받음각 조종면의 변위 엔진출력	모멘트 계수 = 0	연료소비량 최소화 임무거리 최소화

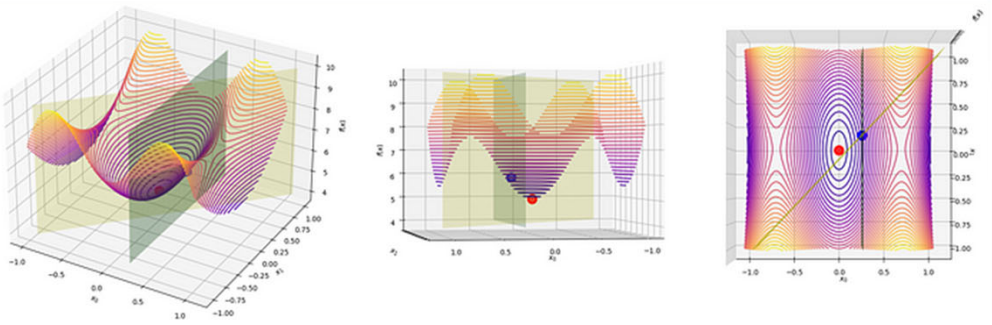
[항공기 개념설계 시 최적화 관련 개념 정리]

SciPy Optimization in Python

- scipy.optimize.minimize 함수 사용하기 (ref.:https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html)
  - ➢ 다양한 method를 제공하고 있으므로, 응용분야에 따라 선택해서 사용할 수 있음
    - ▪ 예제코드 참고해서 실습 필요함 (ref.:https://docs.scipy.org/doc/scipy/tutorial/optimize.html)
      - 추가참고: https://towardsdatascience.com/introduction-to-optimization-constraints-with-scipy-7abd44f6de25
      - 비구속/구속조건 상태에서의 최적화 함수 적용 공부하기
  - ➢ 비용함수의 경우, 그래프로 표현할 수 있다면, 표현하는 것이 기술문서 작성에 큰 도움이 될 수 있음



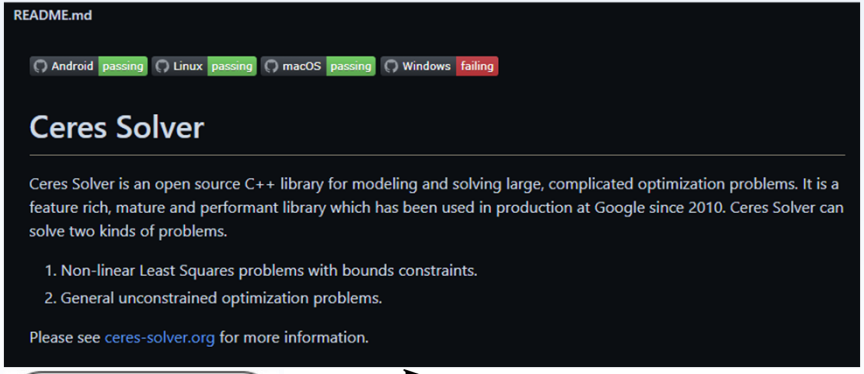
[주어진 비용함수에 대하여 비구속조건 하의 최적화 예시]



[주어진 비용함수에 대하여 구속조건 하의 최적화 예시]

Ceres, ACADO, NLOPT, OPTIM, ensmallen, etc. in C/C++

- 다양한 최적화 라이브러리 존재함
- Toy-problem 을 통해 사용법을 익히면, 상황에 따라 적용 가능
  - ➢ CMake 를 통해 컴파일 시 적용할 수 있음
  - ➢ OpenCV, dLib 등 영상처리 라이브러리에도 최적화 함수 있음

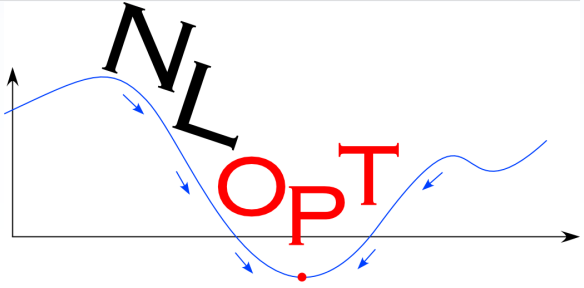


OPTIM

ensmallen

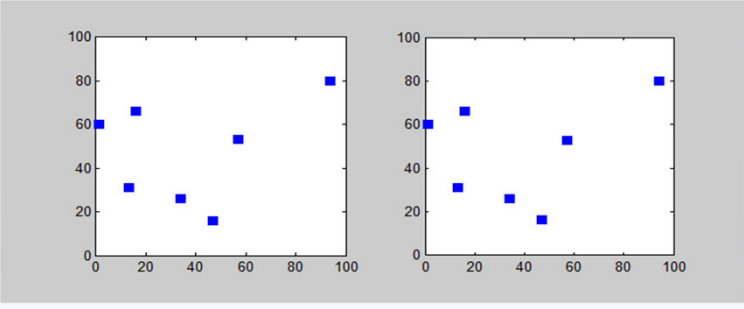
[다양한 C/C++ 기반 최적화 라이브러리]

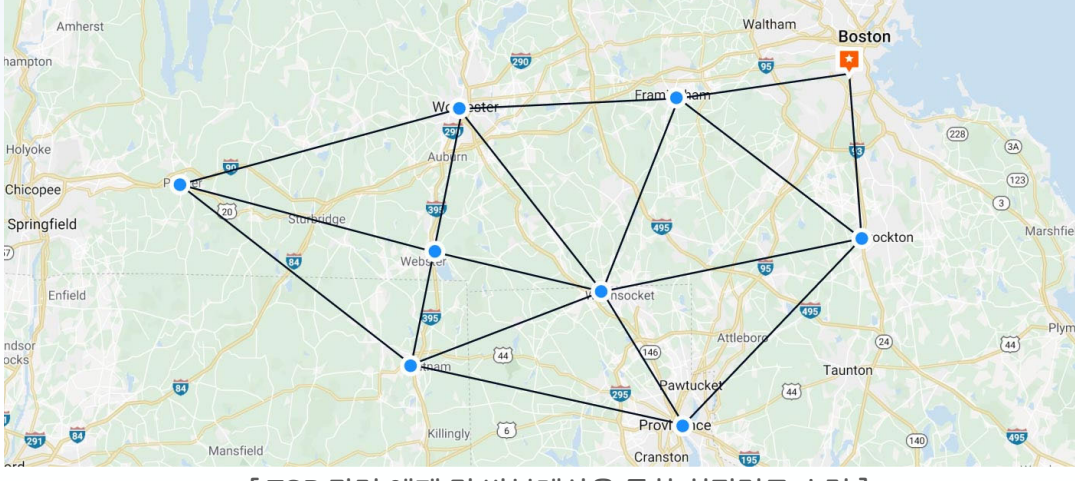
ACADO TOOLKIT



Salesman Problem

- 최적화 학습 시 가장 유명한 문제정의 →조합 최적화 문제(TSP, traveling salesman problem)
- NP-hard problem
  - ➢ 계산 복잡도 이론에 의거, 해를 구하기 어려운 문제의 대표적인 예
  - ➢ 다양한 최적화 문제의 원형이 되고 있음
    - ▪ 그래프 이론에 의거 정의할 수 있음
    - ▪ 대표적인 예시: 회로해석 이후 기판 최적화에 사용되고 있음
      - • 노드 간 거리값에 의거 해석 가능
      - • 코딩 관련 입사문제에 주로 활용
      - • 다양한 풀이 방법 존재





[TSP 관련 예제 및 반복계산을 통한 최적경로 수렴]

일반적으로 적용 가능한 단일 변수 탐색방법 (단순함)

- 이분법(bisection method)와 의미가 유사함
- 제약조건
  - ➢ 초기 구간 내에 한개 이상의 최대/최소값이 포함되어 있어야 함
    - subscript lower(l): 하부 경계값, subscript upper(u): 상부 경계값
- 왜 황금비율인가?
  - ➢ 두 번째 조건부터 길이의 비가 같도록 설정하는 과정 중에 황금비율이 발생함
    - 첫 번째 조건

$$l_0 = l_1 + l_2 \qquad \frac{l_1}{l_0} = \frac{l_2}{l_1}$$

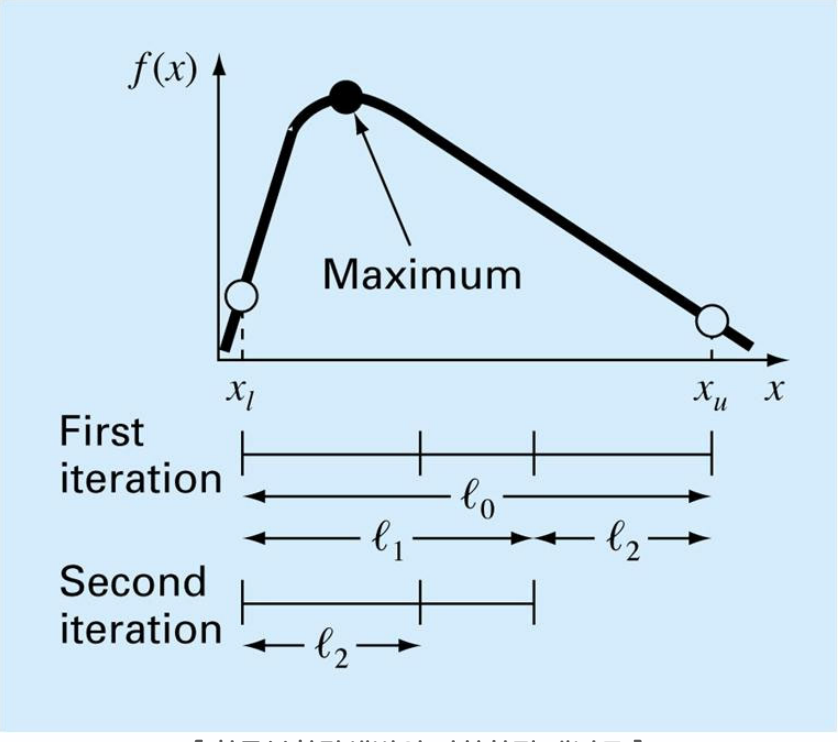
    - 두 번째 조건

$$\frac{l_1}{l_1 + l_2} = \frac{l_2}{l_1}$$

$$R = l_2 / l_1$$

$$1 + R = \frac{1}{R}, \quad R^2 + R - 1 = 0$$

$$\rightarrow \therefore R = \frac{-1 + \sqrt{1 - 4(-1)}}{2} = \frac{\sqrt{5} - 1}{2} = 0.61803$$



[황금분할탐색법의 기하학적 개념도]

알고리즘 개념

- #1: $f(x)$의 지역적 최대/최소값을 포함하는 초기 x_l과 x_u로 시작
- #2: 두 개의 내부점 x_1과 x_2를 황금비를 이용하여 구함
  - ➢ 비율 계산하여 x_1과 산출

$$d = \frac{\sqrt{5}-1}{2}(x_u - x_l)$$

$$x_1 = x_l + d \qquad x_2 = x_u - d$$

- #3: 조건 비교하여 x_l과 x_u을 갱신
  - ➢ $f(x_1) > f(x_2)$
    - ▪ x_2 의 좌측에 있는 영역 $x_l \sim x_2$ 을 제거 (i.e.해당 구간에 최대값이 없음)
    - ▪ x_2를 다음 반복의 갱신된 x_l 로 적용, x_1 을 현재 상태의 x_{opt} 로 적용
  - ➢ $f(x_2) > f(x_1)$
    - ▪ x_1 의 우측에 있는 영역 $x_1 \sim x_u$ 를 제거 (i.e.해당 구간에 최대값이 없음)
    - ▪ x_1 을 다음 반복의 갱신된 x_u 로 적용 , x_2를 현재 상태의 x_{opt} 로 적용
- #4: 오차 계산하여, 지정된 오차보다 작으면 반복 종료, 그렇지 않으면 #1-#3 반복

$$\varepsilon_a = (1-R)\left|\frac{x_u - x_l}{x_{opt}}\right| \times 100\%$$

- 황금비 사용에 따른 이점
  - ➢ 초기 구간 내에 황금비를 통하여 두 개의 내부점 계산 가능
  - ➢ 이후 단계에서는 조건에 따라 한 쪽의 내부점만 갱신하면 됨
    - ▪ 반복수행 시 계산량 적음
  - ➢ 황금비 사용으로 인해 구간이 급속도로 줄어듬
    - ▪ 연산량 적은 상태가 될 수 있음
- 주요 특징
  - ➢ 대형 연산 시 일부로 사용할 수 있음
    - ▪ 함수값 계산 횟수를 최소화하는 것이 매우 유익함
  - ➢ 연산시간이 많이 필요한 경우, 황금분할을 통해, 연산필요 부분을 탐색하여 해당 부분에 걸리는 시간을 줄일 수 있음

황금분할탐색법 코드 작성을 위한 의사코드

```
FUNCTION Gold(xlow, xhigh, maxit, es, fx)
R = (5^0.5 − 1)/2
xℓ = xlow; xu = xhigh
iter = 1
d = R * (xu − xℓ)
x1 = xℓ + d; x2 = xu − d
f1 = f(x1)
f2 = f(x2)
IF f1 > f2 THEN                          IF f1 < f2 THEN
  xopt = x1
  fx = f1
ELSE
  xopt = x2
  fx = f2
END IF
DO
  d = R*d; xint = xu − xℓ
  IF f1 > f2 THEN                        IF f1 < f2 THEN
     xℓ = x2
     x2 = x1
     x1 = xℓ+d
     f2 = f1
     f1 = f(x1)
```

(a) **Maximization**

```
  ELSE
     xu = x1
     x1 = x2
     x2 = xu−d
     f1 = f2
     f2 = f(x2)
  END IF
  iter = iter+1
  IF f1 > f2 THEN                        IF f1 < f2 THEN
     xopt = x1
     fx = f1
  ELSE
     xopt = x2
     fx = f2
  END IF
  IF xopt ≠ 0. THEN
     ea = (1.−R) *ABS(xint/xopt)*100.
  END IF
  IF ea ≤ es OR iter ≥ maxit EXIT
END DO
Gold = xopt
END Gold
```

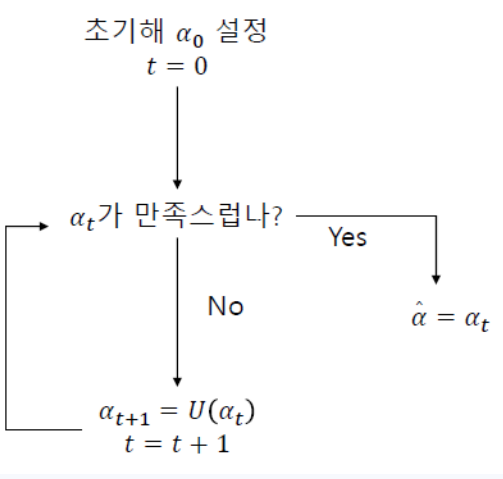
(b) **Minimization**

[황금분할탐색법의 naive code 작성을 위한 의사코드]

(ref.: https://angeloyeo.github.io/2020/08/16/gradient_descent.html)
(ref.:https://bigdatajessie.tistory.com/71)
(ref.:https://velog.io@ysw2946/3.-경사하강법-순한맛)

1차 미분계수를 이용해 함수의 최대/최소값을 찾아가는 반복적인 방법

- 왜 사용하는가?
  - ➢ 실제 정의된 문제들의 함수는 closed form이 아니거나, 함수의 형태가 복잡할 수 있음(i.e., 비선형 함수)
    - ▪ 미분계수와 그 근을 계산하기 어려울 수 있음
  - ➢ 실제 미분계수를 계산하는 과정을 컴퓨터로 구현하는 것에 비해 비교적 쉽게 구현 가능
  - ➢ 데이터 양이 매우 크면, 반복적인 방법을 통해 해를 구하면 계산량 측면에서 더 효율적임

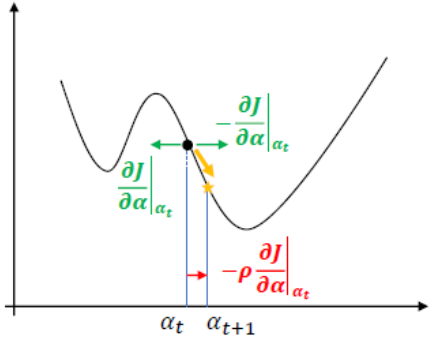


J = 목적함수

$\left.\frac{\partial J}{\partial \alpha}\right|_{\alpha_t}$: α_t에서의 도함수 $\frac{\partial J}{\partial \alpha}$의 값

$$\alpha_{t+1} = \alpha_t - \rho \left.\frac{\partial J}{\partial \alpha}\right|_{\alpha_t}$$

[경사하강법 개념 설명]



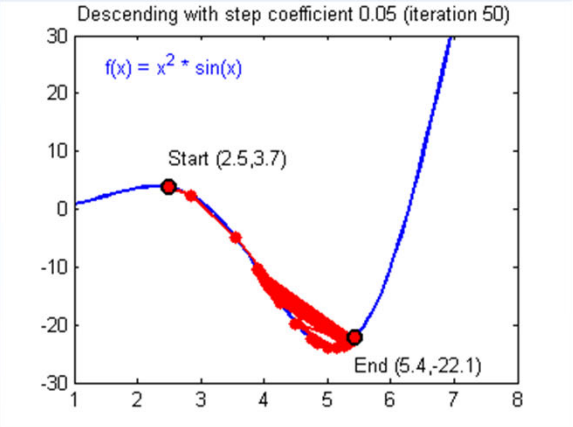
α_t에서의 미분값은 음수이다.

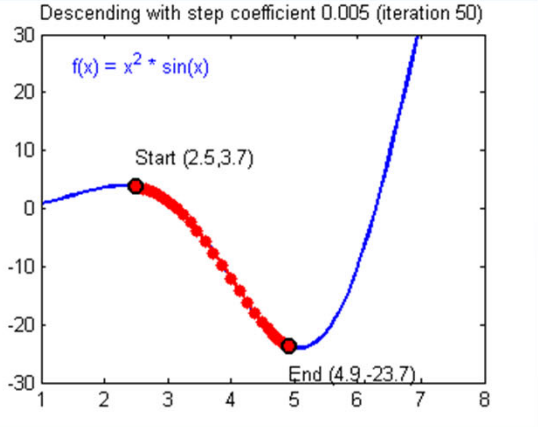
그래서 $\left.\frac{\partial J}{\partial \alpha}\right|_{\alpha_t}$ 를 더하게 되면
왼쪽으로 이동하게 된다.
그러면 목적함수의 값이 증가하는
방향으로 이동하게 된다.

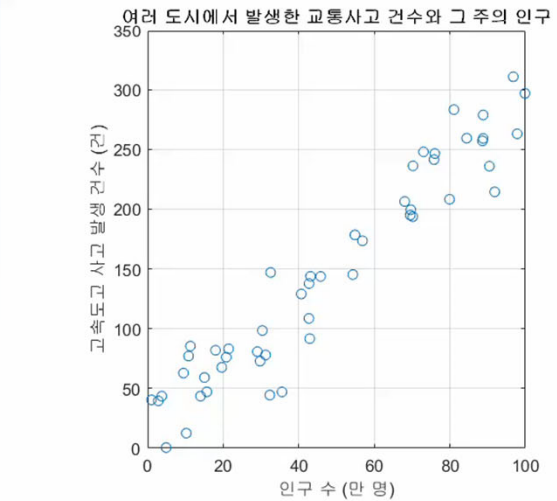
따라서 $\left.\frac{\partial J}{\partial \alpha}\right|_{\alpha_t}$ 를 빼준다.
그리고 적당한 ρ(스텝크기, 학습률)를 곱해주어서
조금만 이동하게 한다.

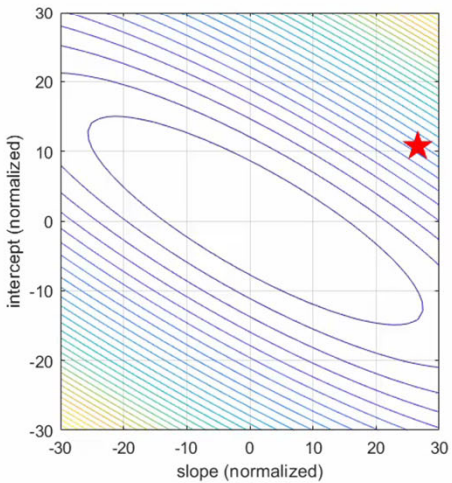
수치해석::비구속 최적화::경사하강법(Gradient Descent Method)

개념 및 수식 관련 설명









$$x_{i+1} = x_i - \alpha \frac{d}{dx} f(x_i)$$

[단일변수일 때의 경사하강법]

$$\mathbf{x}_{i+1} = \mathbf{x}_i - \alpha \nabla f(\mathbf{x}_i)$$

[다변수일 때의 경사하강법]

$$\frac{d}{dx} f(x) = \lim_{h \to 0} \frac{f(x+h) - f(x-h)}{2h}$$

- 함수를 해석적으로 미분할 수 있으면 사용 가능
- 함수를 sympy에 넣어서 미분이 되면 사용 가능
- 둘 다 안되는 경우에는 수치미분 적용해야 함
- 적절한 step size(i.e. learning rate) 설정이 중요함

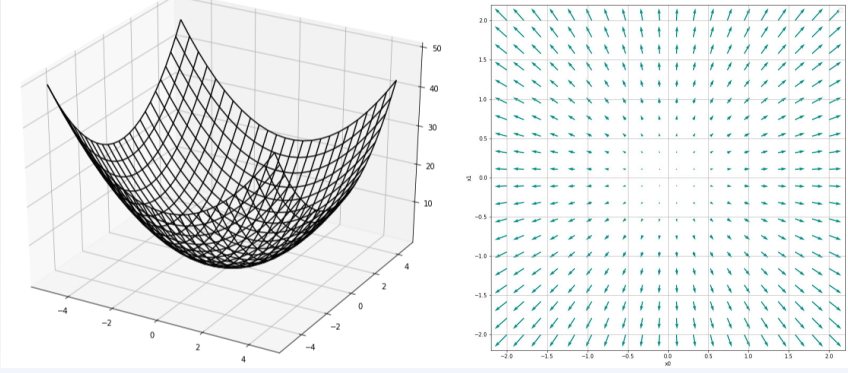
$$f(x) = (x+4)^2 \rightarrow \frac{d}{dx} f(x) = 2(x+4) = 0 \rightarrow \therefore x_{\min} = -4$$

$$f(x) = x\ln(x) - \sqrt{x} \rightarrow \frac{d}{dx} f(x) = \ln(x) + 1 - \frac{1}{2\sqrt{x}}$$

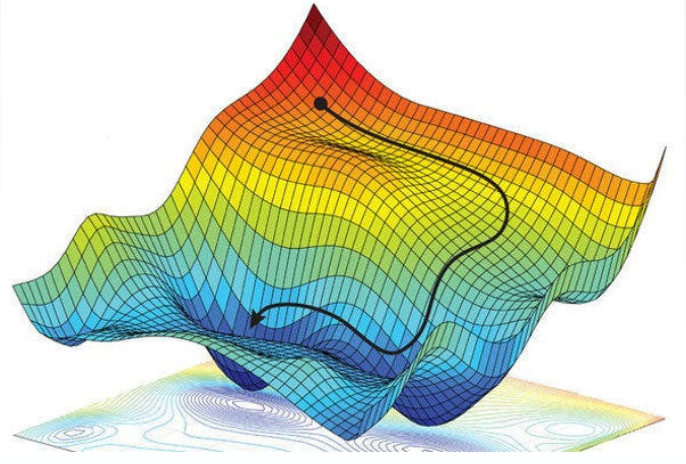
$$\rightarrow \therefore \textit{no closed analytical solution}$$

수치미분을 적용한 경사하강법 예시

```python
def numerical_gradient(f,x):
    h = 1e-4
    grad = np.zeros_like(x) #x와 형상이 같은 배열 생성
    for idx in range(x.size):
        tmp_val = x[idx]
        x[idx] = tmp_val + h
        fxh1 = f(x)

        x[idx] = tmp_val - h
        fxh2 = f(x)

        grad[idx] = (fxh1 - fxh2) / (2*h)
        x[idx] = tmp_val
    return grad

def gradient_descent(f, init_x, lr=0.01, step_num=100):
    x = init_x

    for i in range(step_num):
        grad = numerical_gradient(f, x)
        x -= lr*grad
    return x

def function_2(x):
    return x[0]**2 + x[1]**2

import numpy as np
init_x = np.array([-3.0, 3.0])
print(gradient_descent(function_2, init_x = init_x, lr = 0.1, step_num = 100))
```



[2차 함수 및 그것의 편미분 결과 도식화]



[복잡한 함수에 gradient descent 적용 예시]

초기값을 기준으로 미분 가능한 함수의 최소/최대값을 구하는 알고리즘

- 경사하강법과 유사함
- 비선형 방정식 해를 구하는 것에서 유도할 수 있음
  - f(x)=0을 만족하는 근 x를 찾는 개구간법 정의

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

  - g(x) = f '(x) 로 정의한 후 f(x)의 최적값을 구함

$$f'(x^*) = g(x^*) \rightarrow x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)}$$

- 주요 특징
  - f(x) 에 대한 2차 Taylor 급수를 전개한 후 함수 값의 변화가 0이 되도록 유도
    - Newton법은 최적값을 포함하는 구간에 대한 초기추측값이 필요하지 않음
  - Newton-Raphson법과 유사한 개구간법
    - 문제점
      - 1차 미분도 어려운데 2차 미분까지 알아야 하는 문제 있음 (수치적인 불안정성 큼)
      - 초기값 설정에 따라 발산할 수 있음

	One-dimensional Optimization	Multi-dimensional Optimization
At the optimal	$f'(x_i) = 0$	$\nabla f(\mathbf{x}) = 0$
Newton's Method	$x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)}$ $f''(x_i)^{-1} f'(x_i)$	$\boxed{\mathbf{x}_{i+1} = \mathbf{x}_i - \mathbf{H}_i^{-1} \nabla f(\mathbf{x}_i)}$ $\mathbf{H}_i$ is the Hessian matrix (or matrix of 2nd partial derivatives) of f evaluated at $\mathbf{x}_i$.

[단일/다중 변수에 적용한 뉴턴-랩슨법]

$$H(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

비교적 복잡한 계산, 함수에 따라 수렴성 낮음 → 어떻게 극복하나?

- Hessian matrix 대신 x_n의 주변에서 2차 도함수의 근삿값을 구해 계산한다.
- 준 뉴턴 방법 중에서 BFGS(Broyden-Fletcher-Goldfarb-Shanno) 방법이 많이 사용된다.
  - ➢ Hessian matrix 계산 또는 저장 시 연산량이 과도한 경우 사용
    - ▪ 가장 최근의 iteration들에서의 curvature information을 이용
    - ▪ 반면 오래된 iteration들의 curvature information은 현재 iteration의 Hessian이 보이는 동향(behavior)과 다소 거리가 있으므로 사용하지 않음
  - ➢ scipy.optimize.minimize 기능을 사용
    - ▪ option에서 BFGS 선택하여 적용할 수 있음
- 비선형 방정식의 해를 구할 때와 유사, 단, 입력 파라미터 변경됨
  - ➢ f, f' 대신, f', f'' 를 사용해야 함

LBFGS computation of $p^k = -H^k \nabla f(x^k)$

1. $q := -\nabla f(x^k)$
2. for $i = k-1, \ldots, \min(k-m, 0)$
   $\alpha_i := \frac{(s^i)^\top q}{(y^i)^\top s^i}$
   $q := q - \alpha y^i$
   end for
3. $p := H^{0,k} q$
4. for $i = \min(k-m, 0), \ldots, k-1$
   $\beta := \frac{(y^i)^\top p}{(y^i)^\top s^i}$
   $p := p + (\alpha_i - \beta) s^i$
5. return p

In step 3 $H^{0,k}$ is the "initial" H. Popular choice

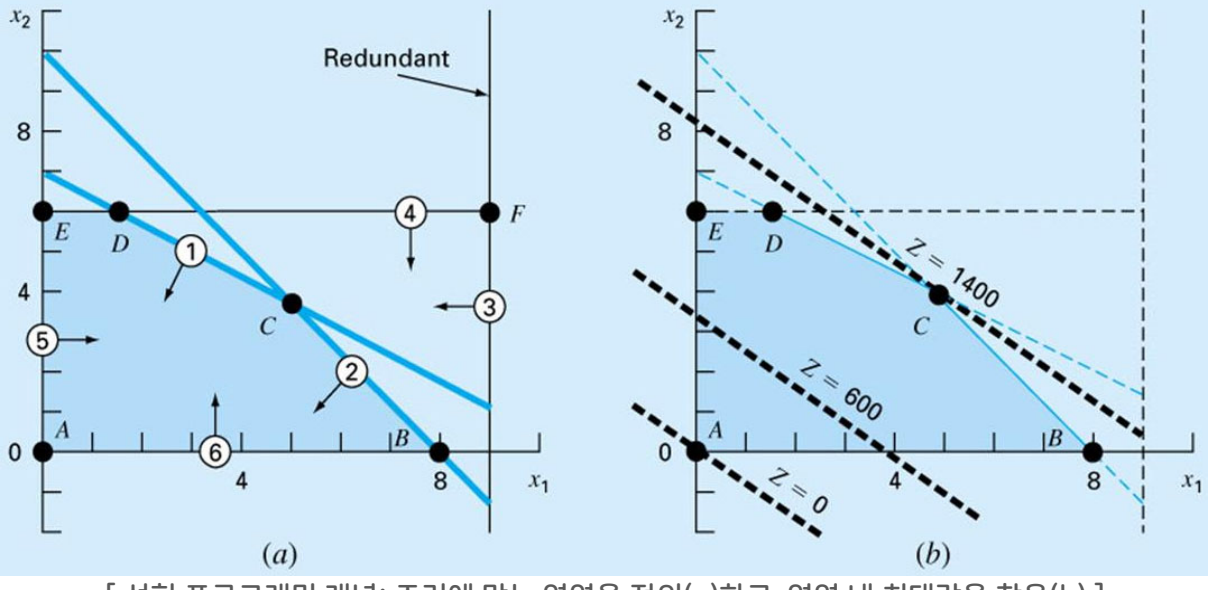
$$H^{0,k} := \frac{(y^{k-1})^\top s^{k-1}}{(y^{k-1})^\top y^{k-1}} I$$

```
procedure Newton(f, f', x, nmax, ε, δ)
integer n, nmax;    real x, fx, fp, ε, δ
external function f, f'
fx ← f(x)
output 0, x, fx
for n = 1 to nmax
    fp ← f'(x)
    if |fp| < δ then
        output "small derivative"
        return
    end if
    d ← fx/fp
    x ← x − d
    fx ← f(x)
    output n, x, fx
    if |d| < ε then
        output "convergence"
        return
    end if
end for
end procedure Newton
```

주요 개념

- 선형(linear)
  - ➢ 목적 함수나 구속조건을 표현하는 수학적 함수가 모두 직선
- 프로그래밍(programming)
  - ➢ "컴퓨터 프로그래밍"을 의미하기보다는 "스케줄"을 결정한다는 의미
  - ➢ "업무 목록"을 정하는 의미 함축

- 주요 사용처
  - ➢ 구속조건 하에 최적의 해를 결정하는 문제
  - ➢ 실제 많은 산업현장에서의 최적화 문제는 선형 프로그래밍 형태
    - ▪ 가성비 (i.e. 이윤 최대화, 비용 최소화)
    - ▪ 제한된 자원 하의 최적의 선택을 하기 위함



[선형 프로그래밍 개념: 조건에 맞는 영역을 정의(a)하고, 영역 내 최대값을 찾음(b)]

기본적인 구성

- 목적함수(objective function)

$$(\text{objective function}) = c_1x_1 + c_2x_2 + \cdots + c_nx_n$$

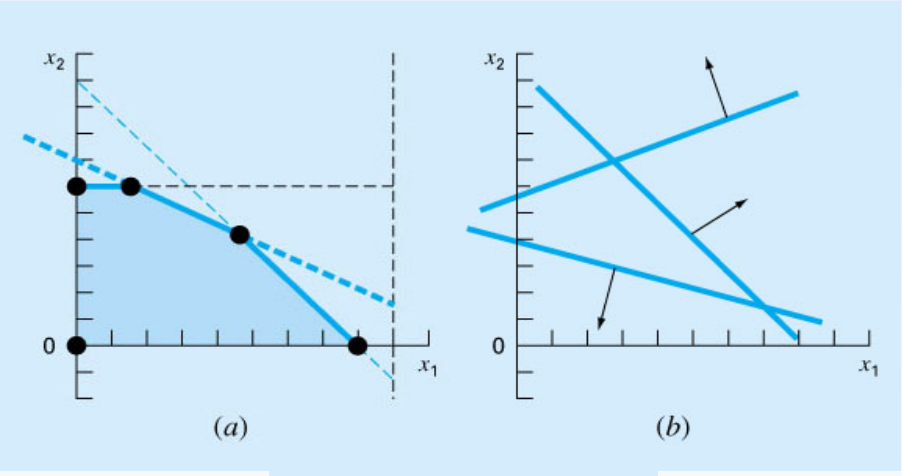
  - c_j는 수행된 j번째 단위활동당 성과
  - x_j는 j번째 활동의 크기
  - 목적 함수 값 Z는 총 n개의 활동에 의한 성과의 합

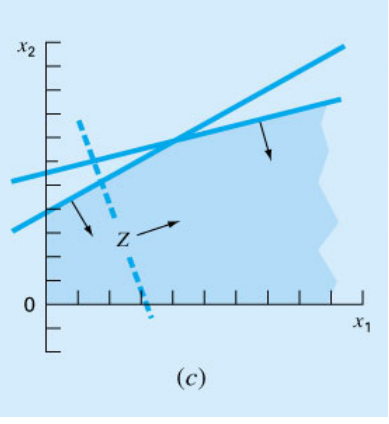
- 구속조건(constraints)

$$a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n \le b_i \qquad x_i \ge 0$$

  - a_{ij}는 j번째 단위활동으로 소요된 i번째 자원의 양
  - b_i는 가용한 i번째 자원의 양(자원은 제한 됨)
  - 일반적인 형태의 구속조건은 모든 활동이 양의 값을 가져야 함

- 해의 상태
  - 유일한 해 존재: 최대값의 목적함수가 한 점에서 교차
  - 여러 해가 존재: 목적함수 직선이 구속조건의 하나와 정확하게 평행
  - 불능해: 해가 존재하지 않음
  - 닫힌 계가 아님: 구속조건 부족





[a: 여러 해를 가질 수 있음, b: 해가 없음, c: 닫힌 계가 아님]

슬랙변수를 도입하여 구속 방정식을 등식으로 재구성한 방법

- 주요 유의사항
  - 풀이 과정 동안에 극점이 일어나는지를 분간할 수 있어야 함
  - 슬랙 변수라고 불리는 것을 도입하여 구속 방정식을 등식 형태로 재구성
- 슬랙변수(slack variable)
  - 구속된 자원의 양이 얼마나 가용한지를 측정
    - 자원의 "잉여(slack)"가 얼마나 가능한지 측정

      $7x_1 + 11x_2 \leq 77$
    - 특별한 생산 상태 (x_1, x_2), 사용되지 않은 상태 슬랙 변수 S_1이라 정의

      $7x_1 + 11x_2 + S_1 = 77$
    - 슬랙 변수가 양수 : 약간의 잉여가 존재
    - 슬랙 변수가 음수 : 구속조건을 초과
    - 슬랙 변수가 0 : 구속조건을 만족

- 완성된 방정식

$$Z\text{의 최대화} = 150x_1 + 175x_2$$

- 구속조건

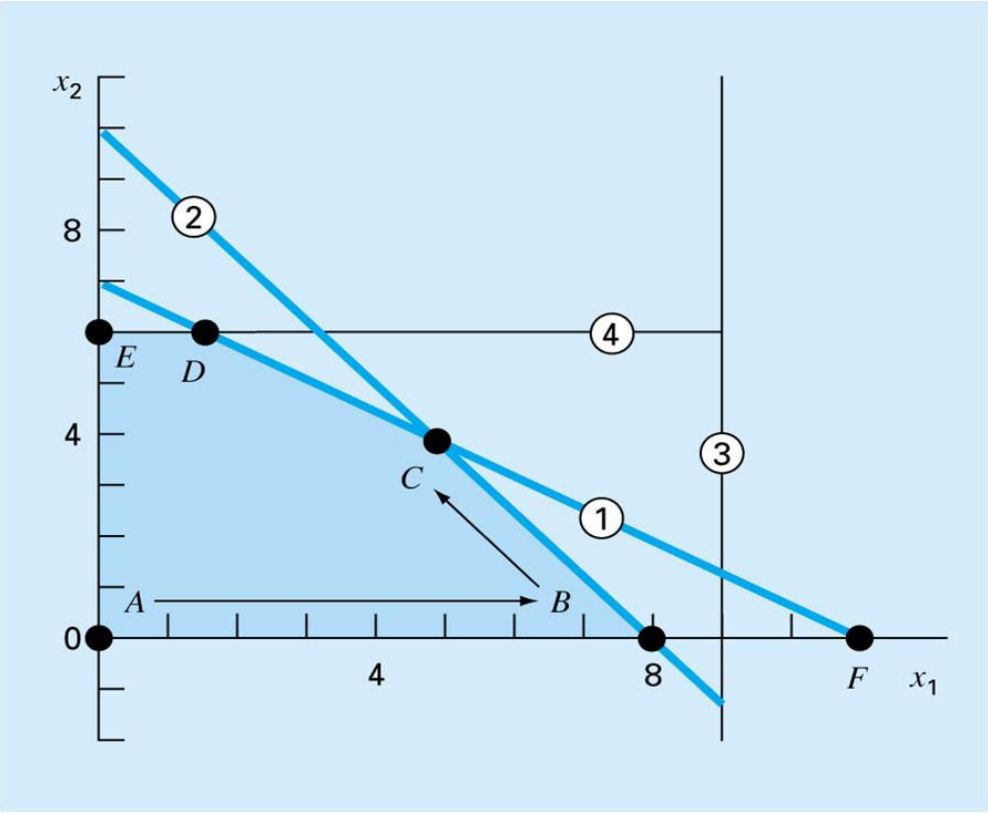
$$\begin{aligned} 7x_1 + 11x_2 + S_1 &= 77 \\ 10x_1 + 8x_2 + S_2 &= 80 \\ x_1 + S_3 &= 9 \\ x_2 + S_4 &= 6 \end{aligned}$$

$$x_1, x_2, S_1, S_2, S_3, S_4 \geq 0$$

[Simplex 문제의 정식화]

정식화 결과의 대수적 풀이 설명

- n개의 미지수를 갖는 m개의 선형방정식으로 일반화
- 기본해 계산
  - n – m개의 변수를 0으로 놓음
  - m개의 미지수에 대한 m개의 방정식을 푸는 것으로 바뀜
    - 비기저변수: 0의 값을 갖는 변수
    - 기저변수 : 남은 m개의 변수
  - 기저변수가 음수가 아니라면 이 결과를 기저가능해라고 부름
- 최적해 계산
  - 모든 기저해를 구하여 어떤 것이 가능한지를 결정
    - 그 해 중 Z 값이 큰 경우를 구하는 것
  - 현명한 방법이 못 됨
    - 적당한 크기의 문제에 대해 수많은 방정식을 풀어야 함
    - n개의 미지수를 갖는 m개의 방정식



[Simplex 문제 해결 과정의 그래프 설명]

- Python SciPy의 linprog 이용 (i.e. 직접 코딩으로 구현하기 어려우므로, 문제 정식화 단계를 통해 라이브러리 적용)
  - ref.: https://docs.scipy.org/doc/scipy/reference/optimize.linprog-simplex.html

항공기 개념설계 (Concept Design of Jet-powered Aircraft)

- 제트엔진 항공기 개념설계 수행 → 정상상태 수평비행을 가정한
  - ➢ 항력 = 형상항력 + 유도항력

$$C_D = C_{D_0} + \frac{C_L^2}{\pi AR}$$

  - ➢ 정상상태 수평비행 상황을 적용한 양력계수 공식

$$L = \frac{1}{2}\rho V^2 S_A C_L \,,\; L = W \;\; (\because steady-state\;\; flight) \rightarrow \therefore C_L = \frac{2W}{\rho V^2 S_A}$$

  - ➢ 항력 공식

$$D = W\frac{C_D}{C_L}$$

  - ➢ 최적화 수행
    - 해수면 위의 10[km] 에서의 최적의 정상상태 순항속도
    - 조건

$$W = 670[kN],\; S_A = 150[m^2],$$

$$AR = 6.5,\; C_{D_0} = 0.018,\; \rho = 0.413[kg/m^3] \;@\; 10[km]$$

  - ➢ 최적화 알고리즘 적용해서 문제풀이 수행
    - 조별 인원마다 각각 다른 방법 1개를 선택하여 수행하기
      - 후보#1: 황금분할탐색법
      - 후보#2: 경사하강법
      - 후보#3: 뉴턴-랩슨법
    - 수행한 결과를 Python SciPy 라이브러리 결과와 비교분석

[항공기 개념설계가 있어야 실제 항공기 제작이 가능함]

지속 가능한 항공연료 만들기 (SAF, Sustainable Aviation Fuel)

- SAF: "SAF는 ASTM D1(미국), Def. 표준 1655-91(영국) 및 CAN/CGSB-91-3(캐나다) 제트 연료 사양, 그 출처는 ASTM D23(합성 탄화수소를 포함하는 항공 터빈 연료)이며 D7566 Jet-A 또는 Jet A-1655로 재식별됩니다. "
  - 친환경 항공류를 적용하여 탄소중립을 실천하려는 국가가 많아지고 있음 (https://www.seoulfn.com/news/articleView.html?idxno=460720)
  - 이를 위해 기존의 항공유에 첨가제를 넣어야 하는 상황임 → 최적의 SAF 만들기가 중요함
  - 주요 조건
    - 첨가제 종류: a_1, a_2, a_3 (금액: a_1=\$0.05, a_2 =\$0.025, a_3 =\$0.15)
    - 전체 첨가제 양이 연료 1.0[L] 당 6.0[mL] 가 되어야 함
    - a_1, a_2 의 합이 3.0[mL] 를 넘으면 안되고, a_1 이 a_2 에 비해 많거나 같아야 함
    - a_3 는 a_2 에 비해 50% 많은 상태여야 함
  - 풀이 필요
    - Python 코드를 통해 그래프 그려서 해석
    - Python SciPy 기반 optimize.minimize 함수 적용
    - Python 기반 Simplex 라이브러리 적용
      - (https://docs.scipy.org/doc/scipy/reference/optimize.linprog-simplex.html)

[항공기에 적용되는 친환경 연료]

QnA